

Codex vs Claude Code，你比的东西就是错的

最近被问了太多次Codex和Claude Code到底选哪个。

每次我都想反问一句，你选的依据是什么？SWE-bench跑分？token单价？每分钟吐代码的速度？

如果是的话，那你比的东西从一开始就错了。

这两个月多我写了五篇Harness Engineering的长文，跟好几个在生产环境里用coding agent的团队聊过，得出一个越来越确定的判断。开发者在选coding agent的时候，90%的注意力花在了模型上，但真正决定你用得爽不爽的，是harness。

<https://markdown.toword.io/zh/tools/markdown-to-pdf>

模型能力每家都在追，差距会越来越小。但harness的设计哲学，决定了你每天怎么跟agent协作。

这两个东西不是一回事。你选一个coding agent，不应该问**它能做什么**，应该问**它能让我用什么方式干活**。

Codex和Claude Code也一样。

大部分开发者选这俩工具的时候，脑子里的问题是「哪个模型更聪明」。但这是个功能层面的问题。真正应该问的是，**这个工具的harness，能让我用什么方式干活，这种方式适不适合我**。

这才是应该比的东西。

我从四个角度把这事拆开说。

第一个角度，你知不知道你在为什么买单

Claude Code的harness做得很深。25个生命周期hook事件，5种权限模式，deny→ask→allow的优先级评估链，分层的CLAUDE.md配置，subagent的工具限制。你可以在agent的每一步行为上插入自己的逻辑。PreToolUse的时候挂个脚本检查命令安不安全，PostToolUse的时候自动跑个lint，SessionStart的时候加载项目特定的规则，PermissionRequest的时候甚至可以调一个外部LLM来做分类决策。Dyad就是这么干的，用Claude Sonnet给Claude Code的权限请求做GREEN/YELLOW/RED三色分类，等于是Claude在监督Claude。我自己在搭agent runtime的时候，就是照着这套思路来的。



还有一个设计我觉得特别爽。auto模式下有一个后台分类器跑在Sonnet 4.6上，判断某个操作能不能自动放行。它看的是用户请求和工具调用本身，不看模型自己生成的那段解释文字。agent不能通过**我写一段特别有道理的justification**来绕过检查。约束层和认知层，从结构上就是隔开的。



但问题是。

大部分开发者不知道这是价值。

你去问一个刚开始用Claude Code的人，**你为什么用这个**，他大概会说**Claude聪明**。他不会说**因为我可以hook在执行层面拦截危险操作**。他甚至不知道hook是什么。他以为自己买的是模型的脑子，其实Claude Code真正差异化的东西是那套可编程的约束层。

产品有价值，但用户说不清这个价值是什么，那它就很难被选中。

Codex的情况正好反过来。它的价值非常好懂，**我把任务扔进去，它在云端跑完了给我看结果，我可以同时开好几个**。任何开发者听到这句话都能立刻想象自己怎么用。异步委派，多线程并行，sandbox隔离。每一个词都直接对应一种干活方式。

Codex的harness没有Claude Code那么深，但开发者一听就懂它能干嘛。这点上Claude Code差远了。

这不是说Codex更好。这是说Codex更容易被采用。因为用户**知道**自己买了什么。

第二个角度，harness的回报周期完全不同

Codex上手极快。你装好，扔一个task进去，几分钟后看到一个diff。哦，能用，不错。这个体验循环短到几乎没有学习成本。你甚至不需要写AGENTS.md，默认配置就能跑起来。

Claude Code不一样。它的回报曲线是J型的，前期你得投入大量时间去写CLAUDE.md、调权限规则、写hook脚本、配MCP服务器。前一两周你可能觉得还没有直接用Cursor快。但如果你坚持配下去，到第三周第四周，整个系统开始形成飞轮，agent越来越懂你的项目，你配的规则越来越精准地拦截那些会出问题的操作，效率开始指数级地往上走。

我自己的体感是，Claude Code大概在第三周开始真正回本。

FIG.3 - HARNESS 回报曲线

Codex 即买即用，Claude Code 第三周才回本

Codex 的回报曲线近乎线性，Day 1 就能产出。Claude Code 前期需要投入大量配置时间，但一旦系统成型，效率呈指数增长。



但大部分人撑不到第三周。

其实就是一个回本周期的问题。Claude Code的harness是长线投资，你必须投入才能收获。Codex的harness是即买即用，上手就有回报。两种模式都没问题，但如果你不知道Claude Code需要时间才能跑起来，你会在第三天就弃坑然后跑去说**不好用**。

Anthropic在这个问题上做得不够。他们应该更清楚地告诉用户，Claude Code的真正价值不是Day 1的体验，是Day 30的体验。

第三个角度，你能不能看到harness在帮你

这是一个很容易被忽视的问题。好的harness在干活的时候是隐形的。

Claude Code的hook拦截了一个危险操作，你看到的只是**agent没执行那个命令**。你不知道是hook帮你拦的还是agent自己决定不做的。你的CLAUDE.md规则让agent写出来的代码符合了你的编码规范，你觉得**Claude挺聪明的嘛**，但其实是你写的规则在起作用。

harness干得好的时候你根本看不见它。

Codex在这方面也有同样的问题。sandbox保护了你的环境不被搞乱，但你看到的只是**任务正常完成了**。你不知道如果没有sandbox会怎样。

看不见的东西，在用户心里约等于不存在。

所以你会看到一个很有趣的现象。开发者讨论Codex和Claude Code的时候，聊的永远是模型能力、输出质量、速度、价格。几乎没人聊harness。因为harness在做对的时候是隐形的，它只有在出问题的时候才会被注意到。

这也是为什么我写了五篇Harness Engineering的文章，试图把这层东西从幕后搬到台前。

第四个角度，你是什么时候意识到自己需要harness的

大部分开发者在第一次用coding agent的时候，不知道自己需要harness。他们以为只要模型够聪明就行了。

然后agent删了一个重要文件。

或者agent把一个没测过的代码直接push到了main。

或者agent在你不知道的情况下调了一个外部API，你的key被泄露了。

踩了这些坑之后，你才开始想，我是不是应该给这个agent设点规矩。然后你才开始研究权限系统、hook、sandbox这些东西。

用户一开始不知道自己需要harness。踩了坑之后才发现，原来约束结构比模型能力更重要。

Claude Code和Codex在这个维度上的处理方式完全不同。

Codex的方案是，你不用发现，我直接替你做决定。macOS上用Seatbelt做内核级隔离，Linux上用Landlock加seccomp-bpf，Windows上用AppContainer。agent phase默认断网，文件系统只能碰你显式给它的项目目录。你什么都不用配，agent从OS层面就被关在笼子里了。对于不知道自己需要harness的新手来说，这其实是更安全的默认值。

Claude Code的方案是，我给你工具，你自己决定怎么用。5种权限模式，Shift+Tab随时切换。default模式每步确认，acceptEdits模式放行文件编辑但卡命令，plan模式只读不改，dontAsk模式只允许白名单操作，bypassPermissions模式全部放行。你觉得太繁了可以开auto，但auto的边界需要你自己通过hook和deny规则去调。这对于知道自己要什么的老手来说，灵活度极高。但对新手来说，他可能连Shift+Tab能切模式都不知道。

一个是保姆模式，一个是工具箱模式。

两种默认值哲学

Codex 替你做决定，适合不知道风险在哪的新手。Claude Code 给你做决定的工具，适合知道自己要什么的老手。



保姆适合不知道风险的人，工具箱适合知道自己在干嘛的人。

把四个角度放在一起看，结论其实很清楚

Codex和Claude Code不是在模型层竞争。它们是两种完全不同的harness哲学，服务的是两类不同的开发者需求。

Codex卖的是**省心**。一句话能说清它干嘛的，上手即用，sandbox兜底。你不需要懂harness，它帮你把安全和隔离都做好了。代价是你没法精细控制agent的行为。

Claude Code卖的是**掌控**。上手慢，配置成本高，但天花板也高。你愿意投入的话，最终能搭出一套完全按你的规矩运转的agent环境。代价是前期投入大，而且harness干得好的时候反而没人注意到它。

选哪个不取决于哪个模型更聪明。取决于你是什么阶段的开发者，你的项目对安全和规范有什么要求，你愿不愿意花时间去构建自己的harness。

或者两个都用。Claude Code在Opus 4.7上开1M token的context窗口，做需要精细控制的本地工作。Codex在GPT-5.4上跑云端sandbox，做批量并行的后台任务。CLAUDE.md和AGENTS.md在同一个repo里并排放着互不干扰，AGENTS.md还是Linux Foundation管的开放标准，6万多个开源项目都认。我自己现在就是这么干的。

最后说一句。

我一直觉得harness engineering是这波AI coding agent里最被低估的技能。大家都在聊prompt engineering，聊context engineering，但很少有人聊怎么给agent设计约束结构。模型能力每半年翻一番，但你写的CLAUDE.md和AGENTS.md，你搭的hook和sandbox规则，这些东西是跟着你走的。模型换了，harness留下来。

